

Cuadernos de Isdefe

Introduction to **SYSTEMS ENGINEERING** in the 21st Century



Isdefe

Introduction to Systems Engineering in the 21st Century

Volume 1

Systems Engineering Series
Cuadernos de Isdefe



AUTHORS

Dr. Alejandro Salado / Adolfo Sánchez Domínguez / Dr. Dinesh Verma

Thomas Allen McDermott / Víctor Ramos del Pozo

Dr. Ronald Giachetti / Juan Carlos Larios Monje

David Long / Belinda Misiego Tejada / Dr. Kaitlin Henderson

Christopher Delp / L. Miguel Aparicio Ortega / Dr. Joe Gregory

Dr. Kaitlynn Castelle / Miguel Ángel Coll Matamalas



Original Title: Introduction to Systems Engineering in the 21st Century

© Isdefe

C/ Beatriz de Bobadilla, 3 -28040 Madrid

www.isdefe.es

First printing: June 2024

ISBN: 978-84-695-7816-2

No comercial

Legal deposit: M-14809-2024

Publisher: Ingeniería de Sistemas para la Defensa de España SA SME MP

Coordinator: Juan Manuel García Montaña

Technical coordinator: Dr. Alejandro Salado

Editor: Dr. Alejandro Salado

Editing and style review team: Juan Manuel García Montaña, M^a del Rocío Manjón Pérez

Book design and layout: Iliana Aguilar Jiménez

Video recording and production: Favorit Comunicación

Printing by Byprint Madrid

Printed in Spain - *Impreso en España*

The opinions contained in this book are the sole responsibility of the undersigned authors. They are not intended to reflect the opinions or point of view of Isdefe as a company.

The editing team has made every effort to obtain the appropriate permissions for all material reproduced in this book. If there is an omission, we ask that you send us a written request to correct the error.

All rights reserved. Reproduction in whole or in part without the permission of the publisher is prohibited ©.



"[A system is] any portion of the material universe which we choose to separate in thought from the rest of the universe, for the purpose of considering and discussing the various changes which may occur within it under various conditions."

J.W. Gibbs

PROLOGUE

Around 30 years ago, Ingeniería de Sistemas para la Defensa de España S.A. S.M.E. M.P. (Isdefe) published the systems engineering “blue books.” The “blue books” were a series of sixteen monographs that captured the state of the art in systems engineering at the time, in a practical and concise manner. The series was a special feat because of two main reasons. First, it conformed the first systems engineering publications that had been ever written in Spanish; and was certainly the largest one in scope, still to this day. Second, and most importantly, it formally brought systems engineering to Spain at a time when, let’s be honest, just a few individuals were even aware of what systems engineering was.

The “blue books” covered a wide range of systems engineering topics, arguably the whole body of knowledge at the time: from General Systems Theory to Systems Analysis. The series was edited by an editorial board (drafting committee) composed by generals from the Spanish Armed Forces, government officials from Spain, and Isdefe employees, and was written by several national and international experts in the different topics that it covered, including authors such as the late Ben Blanchard, Donald R. Drew, and Jezdimir Knezevic, among others. While the “blue books” were really an impressive and unique source of knowledge, just a few people had access to them. The monographs served as the basis for Isdefe to introduce their employees, clients, and partners to systems engineering, and have remained so. (I am very lucky to treasure one copy of the entire collection in my office despite having never been an Isdefe employee!).



But time has passed and, while one could argue that it has only been a bit short of 30 years since then, the systems engineering landscape has significantly changed, both in the size of its body of knowledge and in its widespread and external awareness. Some examples, non-exhaustive, follow:

- The number of active members of the International Council on Systems Engineering (INCOSE) (founded in 1990 and becoming international in 1995) has grown exponentially since the 2000's. A similar trend exists for the number of new members that join INCOSE.
- A plethora of new methods that span the whole system life cycle has emerged. Systems engineering is not anymore just a process or a guide for good engineering, but systems engineers can resort to methods that are dedicated to its activities, such as capturing requirements, architecting systems, and so forth.
- Today, we have technology that is dedicated to supporting the systems engineer; we no longer must remain cornered to paper, word processors, and spreadsheets.
- There are more and more educational opportunities for systems engineers. We no longer have to limit our learning to on-the-job training, but many universities offer graduate programs, several offer PhD programs, and some undergraduate programs. INCOSE has established a formal certification program, and the number of consulting companies offering training and systems engineering publications (papers, books, reports, etc.) keep growing every year.
- People start to spell systems engineering for real. In Spain for example, systems engineering has been traditionally associated with Information Technology (IT). This is still the case for the most part, but since 2014 there is an INCOSE chapter in Spain (Asociación Española de Ingeniería de Sistemas, AEIS), a podcast, and a few events that get organized every year. In just the last 10 years, the amount of Spanish you hear at a systems engineering conference has dramatically increased. And it feels good!

In this new context, Isdefe is emerging again as a national leader in the field and is set to revise and/or complement the “blue books” with a contemporary look at systems engineering. The purpose of this monograph is to present a snapshot of the state of the practice of systems engineering, including topics that are nascent to practice. The monograph is targeted to practitioners in the Spanish government and industry (while hopefully reaching a wider, international audience), who are engaged in the development of engineered systems, both as customers and suppliers, in the defense, security, space, energy, and transport sectors.

The monograph has been developed as an edited collection of chapters to be read as a monolithic piece, with each chapter being written by one or two international experts in the field in tandem with an Isdefe employee. Each chapter can be thought of as an introduction to a topic that could become a full monograph in the future to tackle in detail the specific topics of the chapter. Our intention with this monograph is that after reading a chapter, the reader gains a solid awareness of the state of the practice and is left willing to learn more about that topic to implement it in their organization.

It has been our intention to write the monograph with an outreach style yet aiming at being technically concise and sound. We have tried to limit content that describes visions for the future of systems engineering, opinions about the state of affairs in systems engineering, or 'sales' pitches that are shallow and unsupported by practice or research. Our goal has been to keep the content as factual as possible, without overstating existing capabilities yet without ignoring modern advances. It is important to recognize however that, given the fluid state of the field right now, different organizations will be at completely different points of maturity with regards to the material presented in the monograph; I dare to state that some may not even be mature on the practices described in the old "blue books" yet. Such organizations should not take the content in this monograph as a utopia, but rather as evidence that there is a path for them to develop and mature their systems engineering capabilities.

The monograph contains six chapters:

Chapter 1 presents the current context in which systems engineering is applied in Europe, as well as some of the competencies of the systems engineer of the 21st century. Main topics include the transition from vertical integration to specialization, the complexity of contractual structures, the alignment of objectives across the supply chain, international teams, dual roles of customer/contractor, and market and political constraints.

Chapter 2 presents three aspects of modern and future systems that may jeopardize traditional systems engineering practices: highly cyber physical systems, distributed governance (systems of systems), and learning-based systems and human-machine teaming. Emphasis is given to current practices with doubtful effectiveness for such kinds of systems and the chapter presents current trends on how to address these unique aspects of these new systems.

Chapter 3 presents the need to evolve and adapt systems engineering development models to the current context of engineering projects. The chapter separates the discussion between traditional, dominantly plan-driven approaches to systems development such as the Waterfall and Vee models, and agile development approaches that emphasize responsiveness. Most projects would benefit from both kinds of models, and the authors discuss hybrid approaches and how to tailor the development models to the context of particular projects.

Chapter 4 moves the discussion from systems and processes to technology for the systems engineer. This chapter formally introduces MBSE, and it covers aspects such as the effects of formalizing systems engineering, the divergence and convergence of semantics, and authoring, conducting reviews, and configuration control in model-based environments.

Chapter 5 presents the novel capabilities enabled by digital models and high computational power of current workstations to support system development and integration. Topics include automated generation and evaluation of architectures, set-based design, and intelligent management of the supply chain and integration process. among others.

Finally, Chapter 6 addresses the same aspects as Chapter 5 but applied to deployment, operations and sustainment, and retirement. Topics include the use of digital twins to perform predictive maintenance, Virtual Reality environments to train users, and the use of Artificial Intelligence to define operational strategies, among others.

Personally, it has been an honor to edit this first monograph for Isdefe's new series, and to work with such a talented pool of authors. I feel fortunate that they all agreed to embark on this adventure with us. I am very grateful to Isdefe and their project management team for trusting and engaging me in this initiative, as well as for having always showed continued support.

On behalf of the authors, the project management team, and Isdefe, I hope that you find the reading educative, enjoyable, and useful.

Dr. Alejandro Salado
The University of Arizona





- /Administration
- /Human Resources
- /Legal
- /Accounting
- /Finance
- /Marketing
- /Publicity
- /Promotion
- /Research
- /Business
- /Development
- /Engineering
- /Manufacturing
- /Planning

INTRODUCTION TO SYSTEMS ENGINEERING IN THE 21ST CENTURY

PROLOGUE	7
1. SYSTEMS ENGINEERING IN THE 21ST CENTURY	15
1.1. <i>A traditional perspective to systems engineering</i>	16
1.2. <i>The current European context for systems engineering</i>	18
1.3. <i>Systems engineering of the present-future</i>	24
1.4. <i>Conclusions</i>	29
2. NEW KINDS OF SYSTEMS	35
2.1. <i>Introduction</i>	36
2.2. <i>Evaluation of the effectiveness of traditional SE practices applied to these new kinds of systems</i>	37
2.3. <i>Trends to evolve systems engineering to effectively engineer those types of systems</i>	44
2.4. <i>Conclusions</i>	51
3. EVOLUTION OF SYSTEMS ENGINEERING DEVELOPMENT AND EXECUTION MODELS	57
3.1. <i>Introduction</i>	58
3.2. <i>Plan-driven development models</i>	60
3.3. <i>Evolutionary or agile development models</i>	62
3.4. <i>Comparison between plan-driven versus agile development</i>	66
3.5. <i>Tailoring development models</i>	68
3.6. <i>Merging development and operations with DevOps</i>	70
3.7. <i>Conclusions</i>	71
4. MODEL-BASED SYSTEMS ENGINEERING	77
4.1. <i>Introduction</i>	78
4.2. <i>Necessary (and ideal) elements of MBSE</i>	79
4.3. <i>Models are more than just drawings</i>	83
4.4. <i>MBSE is not a silver bullet; good systems engineering is a pre-requisite for good MBSE</i>	85
4.5. <i>Novel capabilities enabled by MBSE</i>	85
4.6. <i>Adopting MBSE</i>	89
4.7. <i>Conclusions</i>	92
5. DIGITAL TRANSFORMATION IN SYSTEM DEVELOPMENT	97
5.1. <i>Introduction</i>	98
5.2. <i>Enablers of digital transformation</i>	99
5.3. <i>Model-based lifecycle management and qualification</i>	101
5.4. <i>Beyond traceability and lifecycle management</i>	104
5.5. <i>Conclusions</i>	109
6. DIGITAL TRANSFORMATION IN SYSTEM DEPLOYMENT, OPERATION, SUSTAINMENT, AND RETIREMENT	115
6.1. <i>Introduction</i>	116
6.2. <i>Application of digital twin, virtual environments, and digital thread technologies in later system life cycle phases</i>	117
6.3. <i>Considerations for pursuing digital transformation across the life cycle</i>	122
6.4. <i>Conclusions</i>	125
EPILOGUE	131



"This is where most of us are today: trained in some other field and picked up systems engineering as best we could, engineers with on the job training in SE."

A.W. Wymore

Systems Engineering in the 21st Century

Dr. Alejandro Salado, *The University of Arizona* (alejandrosalado@arizona.edu)

Adolfo Sánchez Domínguez, *Isdefe* (asdominguez@isdefe.es)

Dr. Dinesh Verma, *Stevens Institute of Technology* (dverma@stevens.edu)

Abstract

This chapter presents the current context in which systems engineering is applied in Europe, as well as the ideal competencies of the systems engineer of the 21st century. Main topics include the transition from vertical integration to specialization, the complexity of contractual structures, the alignment of objectives across the supply chain, international teams, dual roles of customer/contractor, and market and political constraints.

Keywords

Keywords: systems engineering history, systems engineering evolution, skills and competencies.



1. A TRADITIONAL PERSPECTIVE TO SYSTEMS ENGINEERING

Traditionally, systems engineering has been conceived as the glue that connects disciplines in an engineering endeavor on the one hand, and the user or customer on the other. The systems engineer acts as a sort of technical coordinator, who makes sure that the decisions made by traditional engineers (e.g., electrical engineering, mechanical engineering, etc.) are well balanced and aligned towards a common goal, which is defined at the system level. A common analogy has been that of an orchestra director, who makes sure the harmonies, timings, and volumes played by each instrument are adequately aggregated to deliver a (hopefully) beautiful piece of music [1]. Without such a direction, the orchestra would likely yield a cacophony instead. The need to balance the desires of engineering disciplines has been captured by several people using cartoons such as the one in Figure 1, which caricaturizes the design of a smartphone as desired by different engineering disciplines and the resulting product once those have been balanced with the customer in mind.

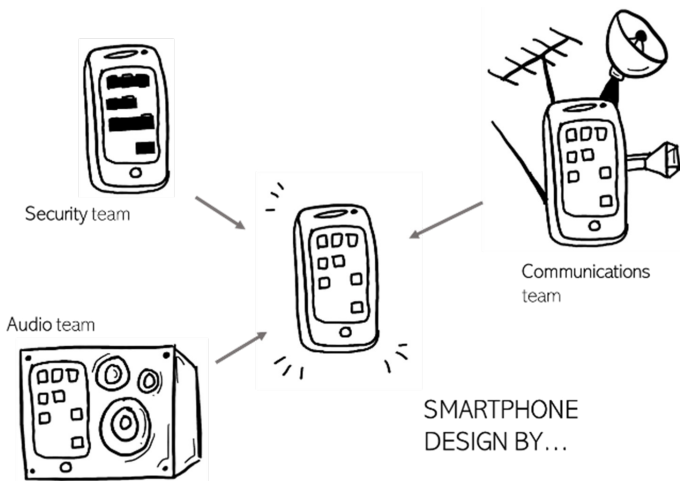


Figure 1. Balancing engineering silos [2]

While technical coordination and integration arguably remains the most common representation of systems engineering, such a view is incomplete though. Lifecycle is the other pillar under which the traditional systems engineering perspective is built. The systems engineer also oversees the evolution of the system throughout its lifecycle (that is, from conception to retirement), and in fact foresees those considerations to inform early decisions; the systems engineer thinks about the end before the beginning. If technical coordination or integration is important to balance the desires of engineering disciplines, being intentional about lifecycle considerations is important to promote consistency between the actual need to be satisfied and the final product that is released into operations. This paradigm has also been often represented by cartoons, such as the one in Figure 2, since at least the 60's.

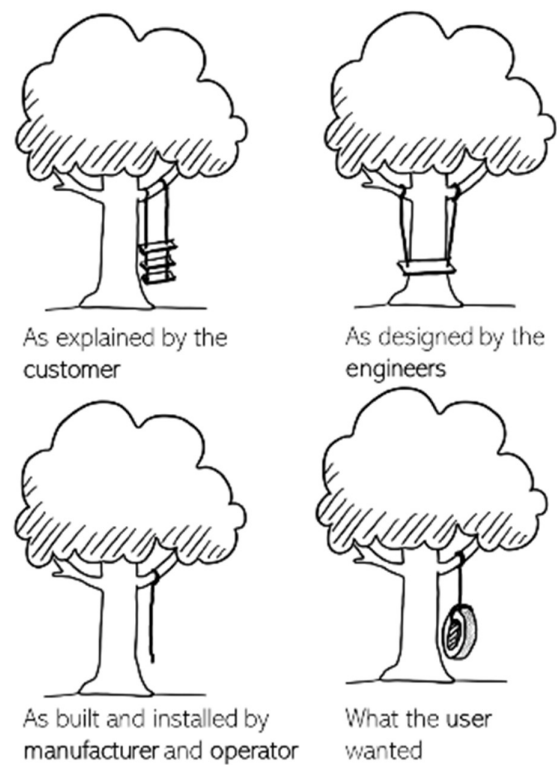


Figure 2. Integration of lifecycle considerations [2]

Several tenets are central to this perspective, which include among others big picture thinking (sometimes equated with a systems mindset), foresight, and communication and influence [3]. Big picture thinking is the ability to identify connections (or relationships) between parts and understand how those relationships yield emergent behaviors at a higher level (usually called the system level). Following the paradigm of the technical coordinator, the big picture thinking is exhibited when an engineer understands, for example, that the power consumption of a given part is not critical in isolation, but that it has effects on the sizing of the power supply, which may affect the required capability of the thermal control system, which may in turn affect the design of the housing structure, which may affect the electromagnetic performance of the system, etc. This applies as well to the paradigm of the lifecycle considerations, where big picture thinking is exhibited when an engineer understands, for example, that a given architecture may achieve an excellent performance but at the unbearable cost of a very intricate integration process or impossible future maintenance.

Probably unsurprisingly, technical coordination in the context of integrating lifecycle considerations is done with the purpose of promoting success in the development of the system. That is, one wants to avoid the unpleasant surprise that the system they developed is infeasible to deploy and operate or even unfit for purpose; this is regardless of whether this is the result of technical inconsistencies or unbearable operations. The ability to foresee and anticipate issues and problems during system development becomes therefore instrumental for a systems engineer. Because of the strong influence that early decisions can have in the success of the development effort (both in terms of development efficiency and solution effectiveness) [4], foresight and anticipation enable steering the development work early on in a way to avoid or easily mitigate the consequences of obstacles that could emerge during the system's lifecycle.

The ability to communicate and influence others is essential for anyone aiming to coordinate the work of other engineers, as well as for anyone working across lifecycle boundaries. For example, a systems engineer may have to convince an antenna engineer that their awesome antenna is unnecessary, and a worse-performing antenna is not only acceptable but what they actually need to yield a feasible solution at the system level. Similarly, a systems engineer may have to convince a thermal engineer that their thermal model does not need so much accuracy, and that just a coarse estimate is sufficient for a particular project. Coordination does not only occur between engineering disciplines though. A systems engineer may have to explain to a project manager why

certain modifications are critical (and do so without resorting to technical jargon!), as well as translate some project management concerns to their engineering team (and do so without appearing to be a manager just cutting down their budget!).

Until very recently and probably in most organizations still today, these skills have predominantly been developed through experience [3] and have heavily relied upon talent [5]. Informally described as scar tissue, the systems engineer possesses a knowledge base acquired through suffering and learning from multiple mistakes and problems in several projects. Every unknown unknown that a systems engineer encounters during his/her career (that is, events that they are unaware of and still materialize), even before they became a systems engineer, is converted into a known unknown that is added to his/her personal set of heuristics or principles (that is, the engineer knows now that the event could occur), acquiring the ability to foresee and act on them before they occur again. Essentially, you've been there, you've seen it, you've lived through it. In addition, a natural byproduct of accumulating such experience is the building up of respect and/or reputation among peers [6]. Generally, these significantly improve one's position when it comes to exerting influence and persuading others [7], and are often associated with career promotions, which afford more opportunities to engage in diverse forms of communication. Furthermore, those individuals with curiosity to stretch beyond one's domain of expertise, can develop their breadth of knowledge as they exchange insights with others in multi- or interdisciplinary teams [8].

Therefore, systems engineering has been traditionally conceived as a step for some senior engineers in their career progression: those that were very good in their respective domains, developed a solid breadth of knowledge across the domains that build up the system of interest, and were able to communicate across silos (including to management, customers, and other individuals engaged in various areas of system development across the lifecycle) would become the systems engineer for a specific project [3]. Systems engineering would not be something you learnt in college, but rather something you would become and grow into.

Given the lack of scientific foundations for the discipline [9], processes became early on an effective vehicle to harmonize systems engineering work [10]. These became so prevalent in systems engineering practice that many started to refer to systems engineering as the systems engineering process, rather than considering it a discipline. Actually, this view is still prevalent in several spheres. (One just needs to search for

the term on the internet, and they will find such treatment by corporations, federal agencies, and universities as of the time of writing this chapter!) Such a process is generally described (simplistically) as a top-down sequence of activities that transform and decompose needs and requirements into a set of components that are later integrated to form a system. This view reinforces the idea of systems engineering as a step in an engineer's career progression, since one only needs to learn and apply a process to an engineering effort.

Given the proliferation of different processes to support a systems engineering effort, such as the Spiral development or more recently Agile methodologies, there has been a trend in the systems engineering community to not consider systems engineering a process itself, but rather a methodological approach: the systems approach. In fact, this is the purview of the International Council on Systems Engineering (INCOSE), which defines systems engineering as "a transdisciplinary and integrative approach to enable the successful realization, use, and retirement of engineered systems, using systems principles and concepts, and scientific, technological, and management methods" [11]. While the approach paradigm certainly encompasses a broader understanding than what the process paradigm did, there is still an underlying implication of systems engineering not being a discipline itself, but something that can be learnt transversally to one's career.

This conception of systems engineering has remained virtually unchanged since the days of the Apollo program. In fact, the Apollo program has remained the main paradigm for the education of engineers for the most part. No doubt, Apollo is probably one of the most, if not the most inspiring and bold engineering endeavors humankind has witnessed. (We certainly believe so.) One may therefore argue that, if such is the case, why not use Apollo as the epitome of how engineers are to be educated, trained, and developed? The answer to that question may be in the fact that it is also fair to acknowledge that the context in which the Apollo program happened may not be representative of the contexts that most engineers face today.

2. THE CURRENT EUROPEAN CONTEXT FOR SYSTEMS ENGINEERING

2.1. Organizational, political, and industrial complexity

As stated, the Apollo program was an outstanding engineering undertaking. But it also benefited from certain contextual features that we do not see often if ever in today's engineering programs. Leaving aside the fact that the Apollo program was a race in the middle of a war (even it was the Cold War), let us look at how NASA's funding at the time compared with today's funding situation (ref. Figure 3).

There are three features to observe. First, one can easily see the peak of funding that NASA received during the Apollo days, which is around double of what NASA received after landing on the Moon. Second, one should note that NASA's budget during the Apollo program was mainly consumed by the Apollo program, while today's budget is spread through many ongoing and future NASA projects. This allocation of resources further amplifies the funding difference between the Apollo program and other NASA programs today (at least as consumed per year). The third aspect, which is probably the most important one with respect to context for engineering practice, was told to the first author of the chapter by a colleague, who supervised a PhD student researching government funding allocation during the Apollo program. Every budget request that NASA solicited to government was approved. That means that, while the funding in today's projects reflects NASA budget (what they have available to spend), the histogram for the Apollo years show what NASA needed to complete the project. Effectively, engineers during the Apollo program did not have cost constraints.

But financial resources are not the only contextual difference between then and now. Organizations adopt one of two primary paradigms to tackle the development of complex systems: vertical integration or horizontal integration. In vertical integration, a single organization owns all stages of the system development process, encompassing conception, engineering, manufacturing, and production. Conversely, organizations adopting horizontal integration rely on different external organizations, which specialize on different aspects of the engineering endeavor, allowing them to leverage their expertise and knowledge. For example, while a vertically integrated organization would design and manufacture every component needed for their system, a horizontally integrated organization would purchase a given component

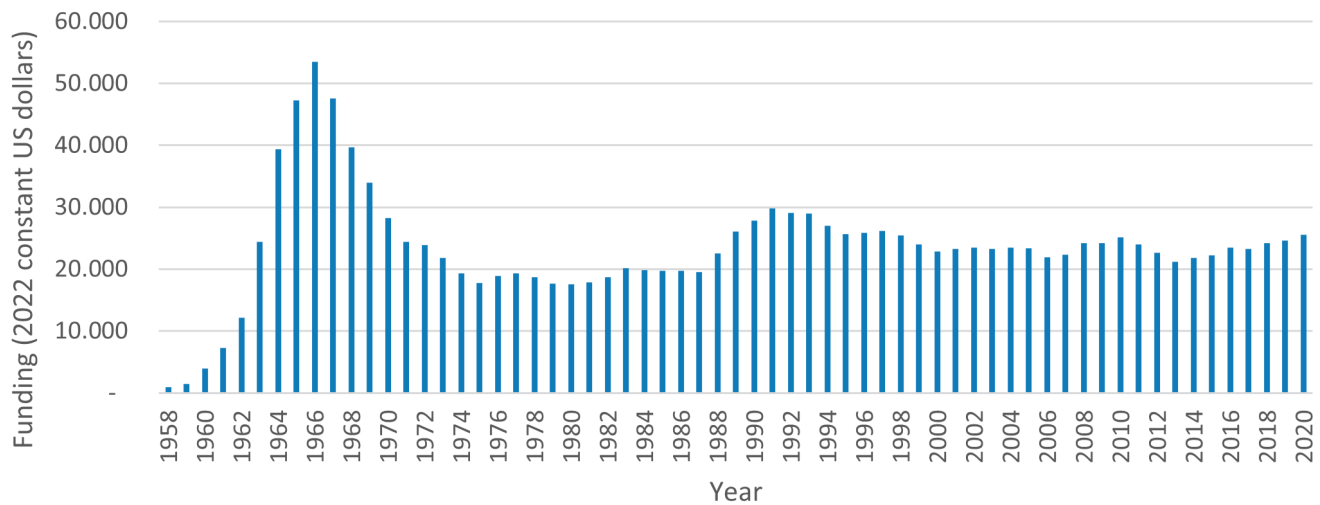


Figure 3. NASA funding over time

(e.g., a battery) to another company that specializes in such kind of technology (e.g., a company that *only* designs and builds batteries). While vertical integration offers greater control over system development, horizontal integration promises higher efficiency due to the allocation of different tasks to expert companies in those tasks. It should be noted though that vertical and horizontal integration should not be understood as binary, but rather a tendency towards more vertical integration or more horizontal integration.

Vertical integration was common in the early days of systems engineering, probably because existing companies would simply grow their existing product portfolio towards products of higher complexity and scale. However, horizontal integration was later favored in the hopes of higher financial efficiency and lower risk promised by specialization and became the norm for large scale engineering endeavors. In fact, horizontal organization is still the most prevalent approach today, particularly in Europe. However, organizations have started to accept that specialization in these areas (i.e., large scale systems such as those in the defense and space sectors) has not lived up to its promise, at least in the way in which it has been implemented.

Horizontal integration introduced the need for contractual structures to govern the relationships between the different organizations engaged in the development of a system (here, the roles of prime contractor, subcontractor, etc. emerge). The burden imposed by contractual constraints often leads to significant costs that, in many cases, have jeopardized any possible gains obtained through specialization¹. Furthermore,

this was an aspect that, while taken into account from a philosophical or conceptual perspective in systems engineering, was not embedded within systems engineering practice in a way that could be operationalized to effectively navigate and/or blend contractual and engineering aspects of an engineering project.

In Europe, this situation is exacerbated by the geopolitical constraints emerging from the need for various countries to not only work together, but to compete and capture European funding. It is not uncommon in European projects that the funding that an organization may receive to pursue an engineering endeavor is proportional and/or bounded by the financial contribution that the country made to the project through the European financial and political channels. This may result from a country's goal to manage a project, to keep control or leadership of a technology, or to develop new capabilities or technologies unavailable to the country at that time, among others. To compete in this context, many multinational companies replicate capabilities in different countries instead of fully implementing specialization, leading to internal competition that confounds with political constraints and competition with other external organizations.

Today, certain organizations are attempting to transition back to vertical integration, in the hopes to integrate their engineering teams more easily towards a common goal. SpaceX has shown with its rockets a notable example of how vertical integration in the 21st century has contributed (albeit not the only contributing factor!) to significantly lower the cost of a large-scale system with respect to horizontal integration.

1. While contracts existed before, the fact that budgets were virtually unlimited made contracting just a vehicle to communicate work, not a source of risks affecting the development effort.

The geopolitical constraints remain though in the European context. While in the USA the political and cultural union with a centralized government facilitates the execution of large-scale engineering programs with a high technological content, in the European context, the fragmentation into multiple states with diverse political and economic interests makes it difficult to tackle large engineering programs that, due to their size or complexity, would be impossible to undertake alone. Supranational organizations (such as the European Space Agency) or international programs (such as Eurofighter, A400-M, or the more recent FCAS (Future Combat Air System)) try to mitigate the effects of this fragmentation by securing stable and prolonged funding commitments from member countries. (The boxed text presents some historical notes and examples of supranational organizations in Europe engaged in the development of large-scale engineered systems.) However, these supranational organizations do not resolve the problems inherent to particular interests of each member country when it comes to the distribution of workloads, the assumption of responsibilities, and the protection of their national industries and security interests.

A clear example of these difficulties is evident in the establishment of the FCAS program, which involves Germany, France, and Spain as the main contributing members. This program faced several challenges. One such challenge was the prioritization of the interests of certain national industries over the needs of the end users. For instance, France, led by Dassault, opposed Airbus serving as the national coordinator for the other two member countries. Additionally, Germany expressed doubts about the project's viability, while Spain preferred Indra over the more international Airbus as its national coordinator. These issues resulted in multiple delays and disagreements among the member countries and their industry partners. Clearly, the fragmented execution of this kind of engineering endeavors incurs enormous transaction costs related to negotiation and decision making among its members. Some of the consequences are the need to offer returns to each of the partners, the different weight of each of them within the programs, the location of production in plants that are sometimes suboptimal, or the need to transport parts between them when all the assembly could be done in a single location. In the end, all this contributes to higher prices for the end customer, excessive development times and, in most cases, delays [12].

Some examples of supranational organizations engaged in the development of large-scale engineering systems in Europe.

In Europe, the treaty establishing the European Economic Community (Rome, 1957) and even the 1997 Treaty of Amsterdam excluded the arms and defense sector from the Community sphere, making it difficult for governments to collaborate and to adopt a common development framework such as might exist in the United States. One of the initial endeavors to foster European cooperation in defense affairs emerged with the establishment of the Independent European Programme Group (IEPG) in 1976. This initiative was conceived as a technical association in alignment with the principles of the Atlantic Alliance, emphasizing the preservation of each member country's national responsibilities. The resolutions produced by the IEPG, though lacking binding authority, primarily served as a mechanism for exchanging information regarding national armament and equipment procurement procedures. Furthermore, they facilitated the investigation and evaluation of potential frameworks for overseeing joint projects.

In 1992, the IEPG became the Western European Armaments Group (WEAG) and was integrated into the Western European Union (WEU) as the body responsible for armaments cooperation. Despite continuing the procedures and relations of the IEPG, it established a series of new objectives such as the search for competition between the different national markets, the reinforcement of the technological base, and cooperation in defense R&D. In 1996, the Western European Armaments Organization (WEAO) was established; a new WEU subsidiary body essentially dedicated to managing research and technology activities.

Given the limited impact of these initiatives, a group of countries began to make progress in this field through different agreements until 1996, when the Organization for Joint Armament Cooperation (OCCAR) was formed, the main organization in the field of industrial cooperation in armaments and the embryo of the European Defence Agency (EDA). OCCAR's Program portfolio currently includes 17 important armament programs with a total operational budget in 2023 of about 6 Billion €: A400M, BOXER, COBRA, ESSOR, FREMM, FSAF-PAAMS, LSS, LWT, MALE RPAS, MAST-F, MMCM, MUSIS, NVC, PPA, REACT, TIGER, and U212 NFS. The governance of OCCAR and the management of the OCCAR programs follow the OCCAR rules.

Meanwhile, in Spain, ISDEFE (Ingeniería de Sistemas para la Defensa de España), a state-owned company, was established in 1985. It was created to serve as a resource for providing technical support in complex projects, with a particular focus on the defense, aeronautics, and information and communications technology sectors. Additionally,

ISDEFE was tasked with offering technical assistance to the Ministry of Defense in systems engineering efforts, particularly related to the Modernization Programs of the Armed Forces. Since its inception, ISDEFE has been actively implementing Systems Engineering methodologies in the development processes of various systems, including command and control systems, air traffic control, platform reengineering, logistics chain optimization, intelligence and electronic warfare systems, and surveillance and border control systems. Notably, ISDEFE has played a crucial role in numerous military programs, such as the F-110 frigates, the VCR 8x8 wheeled armored vehicle, the A400M military transport aircraft, the Tigre helicopter, the S-80 submarine, and many others.

In 2004, the European Defense Agency (EDA) was founded to help its twenty-seven member states (all EU countries) to develop their military resources by pooling national interests and catalyzing the operational, technological, and industrial aspects required to implement multinational weapons systems programs, delegating the actual management of the programs in their development and production phases to organizations such as OCCAR.

In the aerospace field, two initiatives set the European framework for collaboration in systems engineering: the creation in 1975 of the European Space Agency (ESA) and the establishment in 2000 of the aerospace company European Aeronautic Defence and Space (EADS), renamed in 2014 as Airbus Group.

The creation of the European Space Agency (ESA), like that of EDA, was preceded by the creation of other bodies such as the European Space Research Organization (ESRO) in 1962, the European Shuttle Development Organization (ELDO), the European Space Research and Technology Centre (ESTEC), which would be responsible for the development of satellites and space vehicles, and the European Space Operations Centre (ESOC), responsible for the control of satellite operations. In 1973, with the global agreement of all member countries, three projects were approved (Spacelab, the Ariane Program, and Marots) and a fundamental decision was taken: the creation of the European Space Agency (ESA).

The European Commission has sought to overcome the issues of fragmentation by establishing the following initiatives:

- The Directorate-General for Defence Industry and Space (DEFIS), which directs the European Commission's activities in these sectors. In the defense industry field, DEFIS is responsible for maintaining the competitiveness and innovation of the European defense industry by ensuring the evolution of a capable European defense technological and industrial base. In the space domain, DG DEFIS is responsible for the implementation of the EU Space Programme, consisting of the European Earth Observation Programme (Copernicus), the European Global Navigation Satellite System (Galileo), and the European Geostationary Navigation Overlay System (EGNOS).
- The European Defence Action Plan (EDAP) of November 2016, which seeks to promote a strong and competitive European Defense Technological and Industrial Base (EDTIB) based on the European Defence Fund (EDF), main EDAP funding framework. This plan is based on four pillars through which specific actions and programs are implemented in the fields of research and development, enabling European defense supply chains and building a single European defense market.
- The Permanent Structured Cooperation (PESCO), of December 2017, which integrates 26 countries and whose key objectives are the improvement of defense capabilities, cooperation in military operations, and the development of joint military capabilities. Its ultimate goal is to strengthen the Union's strategic autonomy in defense matters.

These initiatives add to the ever-increasing set of regulations imposed by governments and federal agencies that organizations and engineered systems must comply with. Public agencies in Europe frequently navigate rigorous and highly regulated tendering procedures. While these procedures aim to ensure transparency and equal opportunities for bidders, they can sometimes become intricate and time-consuming. This complexity may lead to delays in the selection and contracting of engineering organizations. Moreover, the holistic nature of systems engineering demands bidding for contracts of various types (e.g., supplies, software licenses, consulting, works, etc.) involving multiple suppliers and subcontractors, which adds to the challenge of management and contractual relationships among stakeholders. Additionally, in the case of extensive transnational consortia, country-specific regulations for public procurement, data protection, and intellectual property must be harmonized.

In the context of Spain, for example, Article 99.1 of the Public Sector Contracts Law stipulates that “the object of public sector contracts must be determined. The same may be defined in attention to the specific needs or functionalities that are intended to be satisfied, without closing the object of the contract to a single solution.” This requirement compels organizations to clearly define the functional needs of contracted systems from the outset, preventing the project from leveraging the knowledge of companies during its early phases and, in some cases, the adoption of innovative development models, processes, or approaches, like Model-Based Systems Engineering (MBSE), if these are not mandated by the customer.

Time will tell if these initiatives succeed, but past experiences do not seem to correlate the addition of more governing bodies and regulations with success in system development. Concerns for overreliance on processes have been claimed as early as in 1969 already during the Apollo program, “If I plot a graph versus time of what appears to be a recent rising tide of costs, cost overruns, unsatisfactory performance and unhappiness among engineers, I have reason to worry. [...] If I plot on the same graph versus time the rise in talk, directives ... I see high correlation between the two graphs” [10], and were reaffirmed in 2010 by former NASA Administrator Mike Griffin [5]. In simplistic terms, each new regulation adds a new constraint that the solution must satisfy. And we know that each constraint reduces the solution space, and that a reduction of the solution space generally reduces the affordability of the developed system [13].

International collaboration, essential for major European programs, as described, presents other challenges that span beyond those formally introduced by geopolitical constraints. Some examples are listed below:

- Effective communication between multicultural and multilingual teams is not straightforward. Communication problems stem not only from language barriers but also from cultural nuances in communication styles. For example, what may be considered direct and clear communication in one cultural environment, may be considered disrespectful or confrontational in another. At the same time, multilingual teams may find it difficult to convey technical information accurately and in a way that is understood by all members because of losses of information in translation, as well as decreased cognitive performance due to the increase demand in cognitive load when speaking a second language.

- Coordination of activities involving personnel working in different locations and using different tools and technologies in their local work environment can lead to delays in exchanging information and coordinating activities. In addition, there are legal and security concerns for organizations within large consortia as cross-border data exchange may be subject to restrictions, which can add complexity to information exchange and collaboration efforts.
- Cultural and work habit differences may need to be overcome in order to achieve the ultimate goal of a joint system that meets expectations. The way decisions are made can vary greatly from one culture to another. Some cultures may prefer a consensus-based approach, while others rely on hierarchical decision-making. These differences can affect the effectiveness of the decision-making processes in a collaborative project.

These issues should not be interpreted as stating that international collaboration should not be pursued. Not only is international collaboration necessary to develop some large-scale systems, as explained earlier, but it is also preferable in many situations because of several benefits that it can yield, such as leveraging expertise and knowledge. For example, in the case of the Spanish S-80 submarine program, the Spanish Ministry of Defense contracted with the US Navy and General Dynamics-Electric Boat, which allow them to rethink the program and implement better engineering processes and methodologies, including adopting the NASA Systems Engineering Handbook. The point made in this section is that this sets a new context in which systems engineering must be applied, and for which methods and/or approaches may need to be evolved.

2.2. Engineering and technological complexity: New systems, new methods

We have seen in recent years, and continue to see, the emergence of new kinds of systems that exhibit fundamental differences with respect to those with which systems engineering was born. Traditional systems were characterized by being predominantly hardware-based (software was an isolated *tiny* part in those where there was software at all), monolithic (their capabilities did not depend on the capabilities of other systems), often developed in a green field (no dependency on legacy systems), and lacking agency (high predictability on the command-and-control behavior of the system). Traditional systems are less and less common, if they exist at all. For example, when the first

airplanes were developed, all supporting systems also had to be developed to support the airplane (hence their name). Today, new airplanes are developed within the constraints of the existing supporting systems; support systems are no longer in a supporting role but in an enabling one. In other words, the development of contemporary systems is significantly constrained by their need to interact with many systems that are already in place (i.e., legacy systems).

However, the strongest novelties come from the increase in importance and size of software components in contemporary systems, the transition into distributed governance structures, and the emergence of agency.

In the past, software was limited to some very specific functionalities that were otherwise too difficult to implement in hardware. Software was conceived as a last resort, not a preferred go-to solution. Codes could be entirely reviewed and tested. Today, software drives most of the functionality of existing systems or, at least, most functionalities depend on software. One only needs to look at how cars have evolved; even the handbrake is controlled by software! The increase in the reliance on software, together with the increase in its complexity, challenges many of the assumptions under which systems engineering has been traditionally practiced. Software can be evolved quickly, it can be deployed incrementally, it introduces security vulnerabilities of an unprecedented diversity and potential severity, it is difficult if not impossible to comprehensively test, and it can be deployed and upgraded on the fly during operations, among others.

Traditional systems were monolithic in the sense that they alone could yield their intended capabilities; assuming supporting systems were in place. For example, a television would work fine as long as you could connect it to the power grid and tune its antenna. Today we are living the proliferation of systems of systems (SoS): those that are heavily interconnected with and rely on the capabilities of other independent systems whose primary goal is not to serve in a supporting role. In other words, systems that each have their own purpose are somehow leveraged to yield unanticipated capabilities. In the example of the television before, the purpose of the power grid is to provide energy to household devices and the purpose of the content providers is to provide content to consumers. Hence, in both cases the systems' purposes are to serve the television. However, a smartphone affords you the capability of personal navigation by leveraging signals provided by the Global Positioning System (GPS), even though the GPS's primary purpose is to guide missiles, not to help you find your way

somewhere. Recognizing that not every collection of systems is necessarily a SoS is essential to avoid falling into the trap of calling and treating everything a SoS, as the term is often abused and has become a buzzword. Differentiating both is important because traditional systems engineering practices are likely ineffective and sometimes even infeasible to tackle the unique aspects of SoS [14]. In fact, there are even system attributes or performance metrics, such as availability, that we do not even know how to compute and/or predict for a SoS [15]. Certainly, one cannot simply use traditional systems engineering to engineer and/or integrate a SoS and expect to be successful. The application of systems engineering must be adapted (not tailored!) but, in all honesty, the systems engineering community has only started to grasp how to do so. As a body of practice, we are confident that managerial and governance independence of the constituent systems that form the SoS seem to be the latent factors that must inform the evolution. As the ISO standard to apply systems engineering to SoS indicates, approaches based on command and control assumptions, availability of information (e.g., to support verification), existence of requirements, or guaranteed services may be powerless [14]. Some have even claimed that SoS cannot actually be engineered, but only integrated [16], which makes processes related to requirements and architecture inapplicable. Instead, novel systems engineering methods that are based on persuasion and influence, use of incentives (e.g., mechanism design), opportunistic federations, or blended development and operation are being developed.

If distributed governance and the complexity of an ever-increasing reliance on software were not enough changes to the systems we must work with, we have started to witness the incorporation of agency in cyber-physical systems, mainly through artificial intelligence. The exhibition of agency dramatically changes several core assumptions that are central to systems engineering practice. For example, whereas traditional systems are considered to exhibit invariant behavior, in the sense that the behavior is set once the system is built, intelligent systems are designed to change their behavior as needed.

This challenges the effectiveness of many traditional systems engineering practices, which need to be evolved. For example, and certainly not exhaustively:

- Since an intelligent system may change its behavior between the test environment and the operational environment, tests in test environment may no longer be good proxies to predict system behavior during operation [17].

- If an intelligent system is expected to learn from its experiences, individual systems may exhibit an uncontrolled variability of its behavior with respect to its class or family. As a result, product lines may no longer be able to bound a class of systems [18].
- Intelligent systems have a stable underlying/ supporting functionality (that is, to optimize a reward function) and the main driver of system performance and behavior is the training data. Functional decomposition is no longer able to capture how the system works, since it is data that drives behavior. How does this impact functional-based analyses such as fault-tree analysis or Failure Modes and Effects Critical Analysis (FMECA)?

While some of these problems may be relevant for software-based intelligent systems (such as a function that predicts your purchasing behavior in an online store), their solutions may not be directly transferrable to general systems, because of the highly coupled effects of the physical world. A new evolution of systems engineering is therefore necessary.

3. SYSTEMS ENGINEERING OF THE PRESENT-FUTURE

Traditional systems engineering paradigms and methods are becoming ineffective to deal with the new contexts – organizational and technological – in which engineering endeavors are being undertaken. The role of a single senior engineer trying to coordinate technical efforts by managing information in countless documents, manually controlling configuration items, relying on free-style sketches to model different system facets, and gut-feeling most decisions is slowly fading away. Methods underpinned by science, formal decision making, formal modeling, digitalization of engineering artifacts, and a diversity of systems engineering competencies that can be distributed within a team are shaping the present and future of systems engineering.

An overview of some aspects that are relevant to the contemporary and evolving context of systems engineering is discussed in the following sections. Note that these sections are not intended to provide a vision for the future of systems engineering or a roadmap for its evolution; some proposals can be found in other publications (e.g., [19]).

3.1. Emerging theories and foundations

The origins of systems engineering and its development as an engineering practice in the 1960's were accompanied by several efforts to formalize the discipline. Pioneers such as Wymore (e.g., [20]), Warfield (e.g., [21]), or Mesarovic (e.g., [22]), among others, tried to develop mathematical foundations to support systems engineering practice, which, as stated earlier, was primarily driven by intuition, talent, and eventually processes. However, it is fair to state that their work did not succeed in traversing the academic realm into informing systems engineering practice at the time. For several years their work did not only remain unused but the very interest in discovering the foundations of systems engineering faded away against more applied and readily usable research that focused on developing methods.

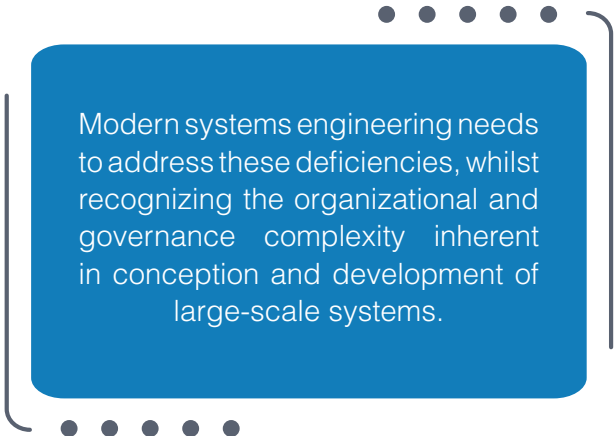
Today, there is a growing recognition, both in academia and industry, for the need of scientific foundations to inform systems engineering practice. Science is a core element of engineering. Absent of scientific principles, we would probably talk of craftsmanship rather than engineering. Certainly, humans built bridges and manufactured products well before Newton formulated the first laws of motion. People resorted to their intuition, heuristics, and experience and were successful with them; here we are today, writing a book chapter on a computer! But we would not say that they practiced engineering. Engineering a bridge or a product is an entirely different feat, since the underlying science enables us not only to understand how things work, but also to use that knowledge to better predict the results of our decisions to improve the effectiveness and efficiency of the products.

By the same token, we should probably be calling systems craft to what we do today, and not use the term systems engineering yet. We have not been able to even agree on or find a rigorous definition for what a system is! [23] Let alone more intricate concepts such as requirements, needs, specifications, architecture, ilities, etc. To move our systems engineering practice, which is built upon experience, gut feeling, good practices, and supposedly good ideas, into real engineering, where we can undoubtedly assess the goodness of a systems engineering method and have a common and consistent set of concepts and associated vocabulary, among others, we need scientific foundations to build upon.

The U.S. National Science Foundation (NSF) has been supporting and funding fundamental research in systems engineering for at least two decades. The Department of Defense (DoD) funded the Systems Engineering Research Center (SERC) 15 years ago as a network of collaborating universities with the goal of transforming systems engineering practice by creating innovative methods, tools, and processes and bringing academia and practice closer together. As of the time of writing this chapter, INCOSE has

launched its Future of Systems Engineering (FuSE) initiative, which includes the Foundations of Systems Engineering as one of its fundamental tracks. While we are far from having a comprehensive and mature set of scientific principles to underpin systems engineering, research has already unveiled several of them, even if most have still not transitioned into practice.

Today, we know that most of the decision methods and/or processes that are used in practice are fundamentally flawed and their recommendations should not be trusted, and why. We know that risk matrices generally embed wrong orders of criticality, and why. We know that most categorizations of requirements lead to poor requirements, and why. We know that verification agreements between customers and contractors should be based on programmatic discussions, not on technical ones, and why. We know that existing MBSE tools are unable to model requirements and that flagging models as requirements leads to poor solution spaces, and why. We know that decisions in engineering should not be consensual in general, and why. We know that verification plans should not be baselined and contracted early in the system development, and why. And, moreover, we also know how many of those activities should be done, and why. The list keeps going but we are not trying to be exhaustive, just indicative.



Modern systems engineering needs to address these deficiencies, whilst recognizing the organizational and governance complexity inherent in conception and development of large-scale systems.

3.2. Systems engineering beyond technical coordination

The conceptualization of the systems engineer as a technical coordinator or technical manager has significantly expanded in recent years. In addition to these, the systems engineers of today can take other roles, including but not limited to requirements engineer, system designer or architect, system analyst, verification and validation engineer, interface engineer, operations engineer, and information engineer (including configuration management and metrics) [24].

Lately, we are even seeing a significant growth of job demands for system modelers (or unfortunately called MBSE engineers), engineers that specialize in the application of MBSE and support traditional systems engineers or systems engineering teams with modeling needs, from converting their ideas into formal models to taking care of model management tasks. In essence, as the field matures, systems engineers may specialize in the different processes or tasks that systems engineering encompasses.

To aid the personal development of the systems engineer, INCOSE has developed a framework that captures the competencies that a systems engineer can acquire and should acquire when targeting certain competency level [25]. The framework categorizes competencies between core (those that all systems engineers should have, such as systems thinking or critical thinking), professional (those that relate to the work of the systems engineer within an engineering team, such as technical leadership or negotiation), management (those related to technical coordination, such as risk management), and technical (those that relate to systems engineering processes, such as requirements or architecture). And it divides expertise between five competency levels, from awareness to expert. An example is shown in Table 1.

A core benefit of the framework is the recognition that a systems engineer does not need to impersonate the knows-it-all role, but that his/her competencies can be developed in line with the systems engineering role that he/she may take. This allows teams to move from every engineer executing systems engineering in their domain (for which strong systems engineering competency cannot probably be developed) to assigning high competency roles within the team, sharing different systems engineering tasks.

Awareness	Supervised practitioner	Practitioner	Lead practitioner	Expert
Describes different types of requirements	Assists with the elicitation of requirements from stakeholders	Elicits and validates stakeholder requirements	Defines and documents enterprise-level policies, procedures, guidance and best practice for requirements elicitation and management processes, including associated tools	Coaches lead practitioners in requirements elicitation and management
Explains why there is a need for good quality requirements	Describes the characteristics of good quality requirements and provides examples	Writes good quality, consistent requirements	Reviews and judges the suitability and completeness of the requirements set	Advises and arbitrates on complex or sensitive requirements-related issues

Table 1. Example of competencies for requirements definition at different levels of competency [extracted from [25]]

Specialization in different systems engineering areas is meaningful. And, while we are going in a good direction, there is still work to be done. For example, it is highly unlikely that someone would assign an electronics engineer the responsibility to perform the structural analysis of an airplane. For such a task, you would choose an engineer that had graduated college with a mechanical engineering degree, had several years of experience conducting structural analyses of growing complexity, and had potentially pursued advanced degrees in structural engineering. However, we find it absolutely sensible to take that electronics engineer and assign him/her the task of writing requirements for a multimillion-dollar system after just offering him/her a 2-day seminar on requirements engineering. Yet, most of the research and reports of most frequent causes of project failure do not list incorrect structural analyses as one of the main causes; eliciting the right requirements right is recurrently listed though (e.g., [26, 27]).

To fill this need, academia is stepping up, both through research (as explained in the previous section) and education. The first undergraduate program in systems engineering started in the early 1960's at the University of Arizona, with a

heavy focus on math, and one of the first master degrees (if not the first one) in the late 1960's at Virginia Tech, with a heavy focus on technical coordination and integration. However, it was not until the mid-2000's that the Stevens Institute of Technology reimaged systems engineering education, with master degrees that focused on the different specialization areas and competencies that are required in modern systems engineering. Such a paradigm has triggered immense growth of master level offerings in the United States of America.

This has been driven by the demands of industry; it is just too risky to rely the future of multimillion dollar ventures on individuals that ignore good systems engineering practices and methods. Formal systems engineering is also slowly making its way into Europe, although maybe too slowly. Systems engineering degrees are rather scattered in different countries, instead of being regularly offered by most universities. For example, as of the time of writing this chapter, only the European University of Madrid offers a postgraduate program in systems engineering in the country, and it is not even a masters degree after the last educational reform. To clarify, it is not the degree that matters, but the competencies and expertise that formal education can

yield. This is something not even discussable for traditional engineering disciplines, and hopefully a future paradigm for systems engineering.

The growth and development of formal systems engineering education at all levels (undergraduate, graduate, and even recently apprenticeship) is also changing how systems engineering exists as a career. If traditionally systems engineering has been a career progression, as described earlier in the chapter, we start to see systems engineering as a career choice itself, where one can progress from junior positions to more senior positions. For example, a junior systems engineer might be responsible for deriving and managing uncritical requirements for the system, while the senior systems engineer can focus his/her attention to just the subject of critical ones.

The emergence and adoption of MBSE is also contributing to this paradigm change, as it is becoming increasingly easier for an organization to recruit MBSE talent directly out of college than train and change the ways of its more senior personnel. In fact, some community colleges have started to offer associate degrees/apprenticeships for system modelers that would take jobs similar to those that drafters had with respect to mechanical drawings.

3.3. Novel methods and tools

The advancement of affordable computational power, together with the formalization of systems engineering (as discussed in section 3.1), have enabled the development of computer-based methods and tools to support systems engineering activities in unprecedented ways. Probably, the most impactful innovation has been the development of MBSE.

In a nutshell, MBSE promotes the use of formal machine-readable models to capture systems engineering artifacts that used to be captured in narrative form or informal models using general office tools (e.g., word processors, spreadsheets, or free drawing tools). In principle, there is no limitation on what systems engineering artifact is captured as a model (e.g., requirements, use cases, functional architectures, physical architectures, or verification plans, among others). But in addition, MBSE also captures the relationships between those artifacts (e.g., allocation of requirements to components, allocation of functions to components, traceability between requirements and their verification activities, etc.). The transition into a model-based environment is expected to provide several benefits, including among others better communication, improved consistency of information,

reduction of development errors, and improved time and cost efficiency [28]. While there seems to be informal agreement within the community of practice about these benefits, it is important to mention that there is a general lack of evidence that these benefits can be actually realized [28].

Today, MBSE is experiencing rapid growth in availability and capabilities of tools, underlying modeling languages, and industry and government adoption that we could only dream of 10 years ago. In this sense, the adoption of Computer-Aided Design (CAD) in mechanical engineering has been often used as an analogy for how MBSE could transform SE. Several organizations are pursuing a similar transition path, opting to acquire software licenses, provide a short training to their employees on tool usage, and let them run with MBSE. However, MBSE is not just a technological evolution, as CAD was [29]. MBSE has implications at the process and methods levels, in addition to the adoption of dedicated computer tools, which also need to mature for the technological adoption to be successful in the long term. In other words, MBSE does not improve poor systems engineering. For example, in terms of modeling, users can now choose between SLD (the proprietary language of Vitech Corporation, Inc.), the Systems Modeling Language (SysML, managed by the Object Management Group and for which its v2 is currently being developed), Capella (an open language developed by Thales), the Object Process Methodology (a language underpinning an ISO standard), and the Lifecycle Modeling Language (LML). These languages or modeling frameworks do not only have different strengths and weaknesses, but their underlying structure has significant implications to the practice of systems engineering. Furthermore, the languages do not offer in most cases a direct translation to guarantee compatibility between them. Failing to realize that the three facets need to be considered for adopting MBSE leads to failure [30, 31].

MBSE adoption has primarily focused on the early development activities, mainly supporting requirements management, system architecture, and change propagation assessment, and is predominantly descriptive. That is, systems engineering models are used to describe aspects of the system (for example, how two components interface with each other) rather than to support quantitative analyses (e.g., evaluating the system level performance given the performance of its components). There are certainly exceptions to this, which have proven the feasibility of using MBSE in later phases of the system development (such as, for example, to plan test and integration activities [32, 33]), and the value of extending descriptive models to perform quantitative analyses [34] or executable simulations [35]. But these are not widely available or employed yet.

Furthermore, MBSE discussions are shifting towards a more general, digital engineering paradigm. In a digital engineering environment, all engineering artifacts are captured in computer models that are semantically connected with each other. This means, for example, that the power consumption attribute in an electronics model is connected with its corresponding power dissipation attribute in a thermal model and with its corresponding power consumption attribute in a power budget system model. This does not imply that all models use the same data and/or values for each parameter. Rather, digital engineering enables the formalization of authoritative sources of truth, which ensure the validity and truthfulness of the data sources. Digital engineering does not invalidate or substitutes MBSE, but it is intended to expand its ideas beyond the scope of systems engineering and throughout the entire lifecycle, from problem formulation to manufacturing to operational support.

Differently than with the adoption of MBSE, which has been facilitated through hybrid top-down and bottom-up efforts (that is, led in some cases by teams of engineers and in some cases supported by corporate leadership or customer mandate) [30], the adoption of digital engineering is being strongly mandated from leadership. The U.S. Department of Defense (DoD), for example, has established a dedicated Digital Engineering strategy [36], which has been rapidly flown down into its service branches. A similar action has been taken by the Ministerio de Defensa in Spain [37]. This approach surfaces a conflict between the urgency with which the customer or organizational leadership desires to advance their capability and the readiness of the community of practice to provide effective solutions in terms of processes and tools. Today, the envisioned capability is still far away from technological readiness.

Computational power has also enabled the acceleration of otherwise time-demanding tasks. Traditionally, the exploration of the solution space during conceptual design or system architecture efforts has been limited to trading-off a handful of alternatives in search of the most preferred solution. However, structured system models can be used now to increase the size of the solution space that can be explored, called tradespace, and automatically evaluate thousands if not millions of solutions *at once* [38]. Sensibly, the larger the solution space that is explored, the more likely it will be to find a better solution. This changes the focus of conceptual design from refining a reduced set of individual possible solutions to construct modular structural models that enable breadth and depth of exploration.

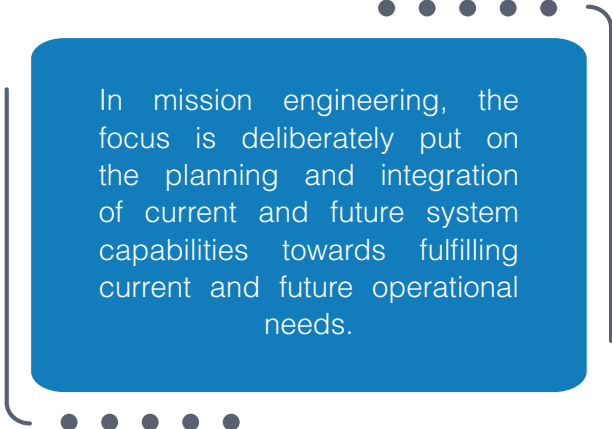
Furthermore, system development has been traditionally limited to a single solution that is chosen and iteratively refined until it goes into production and is later deployed. This means that a system concept or system architecture is chosen, and then it goes through detailed design, where iterations may occur, as the solution is refined into a working one with sufficient maturity to be manufactured and eventually deployed.

This paradigm has been called point-based design, since the design is based on choosing a single solution (point) in the solution space early on. However, computer models can significantly reduce the effort that it takes to maintain and refine a solution. The novel paradigm of set-based design proposes to leverage this advantage to avoid anchoring to a specific solution early in the system development [39], when knowledge is limited [4]. In set-based design, a set of solutions (points) in the solution space are chosen and refined together. Unattractive solutions are only discarded from the set once there is sufficient knowledge to do so with sufficient confidence. This provides flexibility in the development process without requiring changes to solutions, as acceptable solutions are retained.

The way in which past experiences and organizational knowledge may be used in systems engineering is also dramatically changing thanks to the use of machine-readable models and computational support. Cognitive assistants are machines (usually software systems) that aid humans in cognitive tasks [40]. Amazon's Alexa or Apple's Siri are good examples of cognitive assistants that we have started to use frequently, and ChatGPT is further amplifying the capabilities that this kind of systems can provide to humans. A key aspect is that the engineer can use natural language to dynamically and iteratively develop systems engineering artifacts with the support of the assistant. In systems engineering, for example, we have now the capability of not only asking a cognitive assistant to develop conceptual solutions or systems architectures for a given set of parameters, but also to explore the tradespace in several directions and explain its choices and recommendations [41-43].

The evolution of methods and tools has not been limited to those enabled by advances in computational capabilities though. We are also witnessing the emergence of new development processes, models, and paradigms that have emerged out of necessity to cope with the peculiarities of the new context and kinds of missions in which and to which systems engineering is applied now, as discussed in the previous sections.

Development processes that are more iterative, less scaffolded, and more rapid to field systems are becoming more and more frequent (e.g., agile, DevOps) and integrated with more traditional ones (e.g., Waterfall, Vee). Considerable progress has been made to develop the notion of systems of systems engineering, which has been lately redefined as *mission engineering*.



In mission engineering, the focus is deliberately put on the planning and integration of current and future system capabilities towards fulfilling current and future operational needs.

The main distinction here is that those system capabilities are provided by independently governed and/or managed systems.

This discussion is not exhaustive. There are other advances in systems engineering that are probably worth noting, but these should provide an idea of what the landscape of the current practice of systems engineering is (even if still futuristic for some organizations).

4. CONCLUSIONS

The systems engineering landscape has changed in recent years, and seeds have been planted to promote further changes in the years to come. This chapter has hinted at what those changes are and why they have been or are necessary. Resources are not unlimited anymore; rather, they are generally scarce. Systems engineering endeavors trespass geopolitical boundaries and must balance the consequences of industrial consolidations, rapidly evolving technologies, and the emergence of agile startups and rapidly changing market needs. The nature of the systems we must develop and work with now has also changed. Most developments are no longer in a green field, and we must often have to reconcile the constraints of large legacy systems with novel aspects related to increased cyber nature of systems, distributed governance, and even intelligence.

The systems engineer must outgrowth themselves from just being a technical coordinator to a systems engineering expert. Growing into a systems engineer on the job is not sufficient to effectively deal with today's systems engineering efforts. The systems engineer must learn novel methods and techniques, many of which start to be underpinned by research; many good practices have turned out to be not that good and many mature processes are no longer relevant. Technology is taking the center stage of the systems engineering effort, starting with the provision of digital assets that can connect across domains and moving into the use of cognitive assistants to augment the capabilities of the human engineer. While still nascent, these capabilities are rapidly evolving.

Systems engineering has arrived in the 21st century, and it will only keep maturing and evolving. Is your organization ready to modernize its systems engineering practices?

REFERENCES

1. Ryschkewitsch, M., D. Shaible, and W.J. Larson, The art and science of systems engineering. Systems Research Forum, 2009. 03(02): p. 81-100.
2. Salado, A., Systems engineering, in The Engineering Management Handbook, B. Mesmer, et al., Editors. 2023, The American Society of Engineering Management: Huntsville, AL, USA. p. 361-370.
3. Pyster, A., N. Hutchison, and D. Henry, The Paradoxical Mindset of Systems Engineers. 2018, Hoboken, NJ, USA: John Wiley and Son, Inc.
4. Blanchard, B.S. and W.J. Fabrycky, Systems engineering and analysis. Vol. 4. 1990: Prentice Hall New Jersey;.
5. Griffin, M.D., How do we fix systems engineering?, in 61st International Astronautical Congress. 2010: Prague, Czech Republic.
6. Zinko, R., W.A. Gentry, and M.D. Laird, A development of the dimensions of personal reputation in organizations. International Journal of Organizational Analysis, 2016. 24(4): p. 634-649.
7. Manzoor, E., et al., Influence via Ethos: On the Persuasive Power of Reputation in Deliberation Online. Management Science. 0(0): p. null.
8. Delicado, B.A., A. Salado, and R. Mompó, Conceptualization of a T-Shaped engineering competency model in collaborative organizational settings: Problem and status in the Spanish aircraft industry. Systems Engineering, 2018. 21(6): p. 534-554.
9. Collopy, P.D. Systems engineering theory: What needs to be done. in Systems Conference (SysCon), 2015 9th Annual IEEE International. 2015.
10. Frosch, R.A., A new look at systems engineering. IEEE Spectrum, 1969: p. 24-28.
11. INCOSE, Systems Engineering Handbook. A Guide for System Life Cycle Processes and Activities. 5th ed. 2023, Hoboken, NJ, USA: John Wiley and Sons, Inc.
12. Villanueva, C.D., El Programa FCAS y la Industria Española de Defensa: una apuesta equivocada, in Revista Ejércitos. 2022.
13. Salado, A. and R. Nilchiani, A Research on Measuring and Reducing Problem Complexity to Increase System Affordability: From Theory to Practice. Procedia Computer Science, 2015. 44: p. 21-30.
14. (ISO), I.S.O., Systems and software engineering — Guidelines for the utilization of ISO/IEC/IEEE 15288 in the context of system of systems (SoS). 2019.
15. Salado, A. Abandonment: A natural consequence of autonomy and belonging in systems-of-systems. in System of Systems Engineering Conference (SoSE), 2015 10th. 2015.